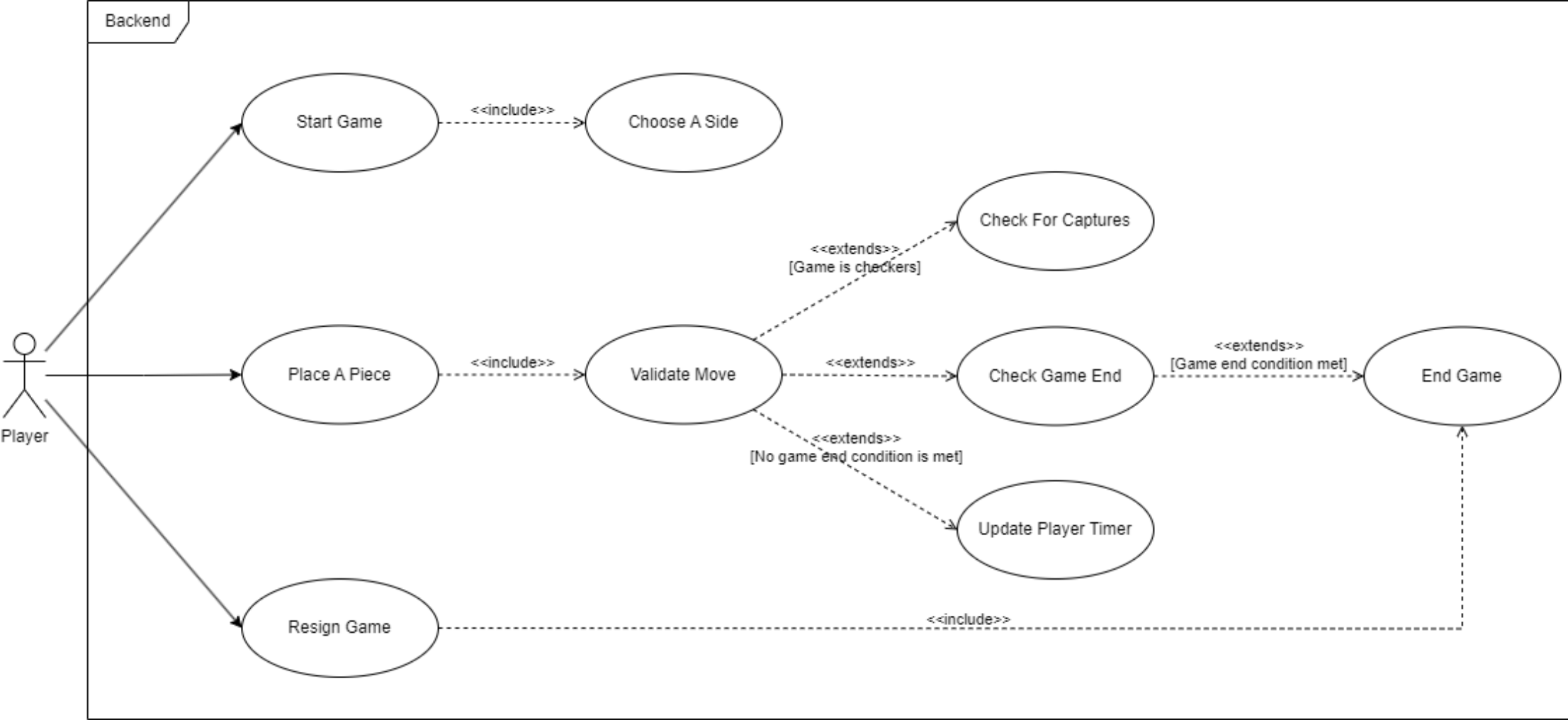

Backend
Start Game
<<include>>
Choose A Side
Check For Captures
<<extends>>
[Game is checkers]
Player
Place A Piece
<<include>>
Validate Move
<<extends>>
Check Game End
<<extends>>
[Game end condition met]
End Game
<<extends>>
[No game end condition is met]
Update Player Timer
Resign Game
<<include>>

Use Case: Check Game End (Win/Lose/Tie)

Iteration: 1

Primary Actor: System

Goal in Context: Determine if the game has reached an end state after each move, based on win, lose, or tie conditions.

Preconditions:

1. The game is in progress with an active session.
2. A player has completed their move and placed a piece on the board.
3. The system has updated the game state with the latest move.
4. Conditions for winning, losing, or a tie are defined and accessible in the game logic (e.g., three in a row for Tic-Tac-Toe, four in a row for Connect 4, or no moves left in Checkers).

Trigger: A player completes a move.

Scenario:

1. The system updates the board with the player's move.
2. The system evaluates the board state to determine if any end-game conditions are met:
    - **Tic-Tac-Toe**:
        - The system checks for three identical pieces in a row, column, or diagonal to identify a win.
        - The system declares a tie if the board is full and no winning condition is met.
    - **Checkers**:
        - The system checks if one player has no remaining pieces or valid moves, declaring the opponent the winner.
    - **Connect 4**:
        - The system checks for four identical pieces in a row, column, or diagonal to determine a win.
        - The system declares a tie if all columns are filled and no win condition is met.
3. If an end-game condition is met, the system concludes the game and declares the result (win, lose, or tie).
4. The system notifies both players of the game outcome.

Postconditions:

1. If a win, lose, or tie condition is met, the game session is ended and the result is recorded.
2. Else no end-game condition is met and the next turn continues.

Exceptions:

- If an error occurs in evaluating the board state (e.g., system glitch), the system resets the board to its last valid state and prompts the player to attempt the same or new move again.

Priority: High, as identifying game-end conditions is crucial for determining the game's outcome.

When Available: 1st iteration.

Frequency of Use: Once per game at the end of the game.

Channel to Actor: The system automatically verifies players via the game interface.

Secondary Actors: Both players in the game session.

Channels to Secondary Actors: The result is displayed on each player's screen at the end of the game.

Open Issues: None.

Use case: End Game

Iteration: 1

Primary actor: Player

Goal in context: Notify the network that the game has ended.

Preconditions: A game is currently in session

Trigger: A final move is made by the player that terminates the game.

Scenario:

1. Starts a game
2. Several moves are made by player(s)
3. A move is made by a player that ends the game
4. Network is notified that the game has ended due to a terminating play

Postconditions: Data is cleaned and reset for next game.

Exceptions: N/A

Priority: High priority

Frequency of use: High frequency

Channel to actor: Mouse

Secondary actors: Network

Channels to secondary actors: Server

Open Issues: N/A

Use Case: Place Piece

Iteration: 1

Primary Actor: Player

Goal in Context: Allow the player to place a game piece on the board during their turn.

Preconditions:

1. The player is logged into their account.
2. The player is connected to an active game session.
3. It is the player's turn to make a move.
4. The player has selected a specific grid location on the game board.

Trigger: The player decides to place a piece on the game board during their turn.

Scenario:

1. The player selects an available spot on the game board by clicking or tapping.
2. The system does a verification check that it is the player's turn.
3. The system checks if the move is valid.
4. The system processes the piece placement based on the game type:
    a. Tic-Tac-Toe
        i. The player selects a square on the 3x3 grid.
    b. Connect 4
        i. The player selects a column in the grid.
        ii. The system drops the piece to the lowest available row in that column.
    c. Checkers
        i. The player selects a valid move for their checker piece, following standard movement rules.
        ii. The system verifies for a capture
5. The system updates the game state and checks for any winning or game-ending conditions.
6. The system notifies the opponent that it is now their turn.

Post Conditions:

1. The game board is updated with the newly placed piece.
2. If a winning condition is met, the game concludes and a winner is declared.
3. Else the turn is handed over to the opponent.

Exceptions:

- The player attempts to place a piece outside the boundaries of the board.
- The player selects an occupied spot (e.g., in Tic-Tac-Toe).
- The player makes an invalid move for the selected piece (e.g., in Checkers, pieces can only move diagonally on dark squares).

Priority: High, as placing a piece is a core action in each game.

When available: 1st iteration

Frequency of Use: Multiple times per game, depending on game length.

Channel to Actor: The player interacts through the screen by tapping or clicking on the board.

Secondary Actors: Opponent player.

Channels to Secondary Actors: Updates appear on the opponent's screen to reflect the placed piece and turn change.

Open Issues: None.

Use case: Start Game

Iteration: 1

Primary actor: Player

Goal in context: Customer starts a game of Tic-Tac-Toe, Checkers or Connect 4.

Preconditions: Player has already selected the game they would like to start.

Trigger: Player clicks the start icon/button on screen.

Scenario:

1. Player opens application
2. Player selects a game they want to play
3. Player clicks start

Postconditions: Game commences, and the player can make a move.

Exceptions: N/A

Priority: High priority

Frequency of use: High frequency

Channel to actor: Mouse

Secondary actors: N/A

Channels to secondary actors: N/A

Open Issues:

- The game the player is interested in may not be available
- The player may want to continue an existing game

Use Case: Update Player Timer

Iteration: 3

Primary Actor: System

Goal in Context: Update the respective timer for each player

Preconditions:

1. A game session has started.
2. The first player makes a move.

Trigger: Either when a game starts, or a player makes a move and changes the turn to the other player.

Scenario:

1. After the game starts, the timer for the first player starts going down.
2. If the first player makes a move, the system:
    - Stops the countdown for the first player timer
    - Stores the timer for Player1
    - Starts the countdown for the second player timer
3. If either of the timers reach an end, the system notifies a corresponding winning message (one player wins).

Postconditions:

1. If the timer ends, the system is notified and the game ends with the corresponding winner being displayed
2. Otherwise, if both timers are still running, the game continues normally.

Exceptions: N/A

Priority: Medium, as the game can work without such functionality. This would help to

increase the complexity of the games, but it’s not mandatory for testing.

When Available: 3rd iteration.

Frequency of Use: Multiple times throughout the game.

Channel to Actor: The system checks both timers’ status automatically.

Secondary Actors: Both players in the game session.

Channels to Secondary Actors: Both players’ respective timer are included in the visual interface

Open Issues: None.

Use case: Resign Game

iteration: 1

primary actor: Player

Goal in context: The player resigns, resulting in a loss and concluding the game.

Precondition: The player is currently in an active game.

Trigger: The player wishes to conclude the game immediately.

Scenario:
1. Player wishes to end the current game
2. Player pressed the resign game button
3. game ends and player loses

Post conditions: The game ends and the player loses

Exceptions:
1. The resignation button malfunctions
2. network issue or player disconnects prior to resigning
3. player attempts to end game after game has already ended

Priority: High

When available: 1 iteration

Frequency of use: infrequent - Expected when players want to end the game prematurely and lose.

Channel to actor: Player's finger has physical interaction with the resign game button

Secondary actors: System

Channel to secondary actors: System receives feedback that game wants to be ended

Open issues:

- **How to handle user misinput:** Should we consider adding a confirmation pop-up to avoid unintended resignations?

Use case: Choose a Side

iteration: 1

primary actor: Player

Goal in context: The player chooses a side.

Precondition: The player started a game.

Trigger: The player starts a game and wants to pick a side.

Scenario:

1. Player starts a game
2. player selects colour or shape from available colours or shapes
3. game begins and other player moves

Post conditions: The game begins and other player moves

Exceptions:

1. The colour or shape malfunctions
2. The game times out

Priority: High

When available: 1 iteration

Frequency of use: Frequent - Every time a new game is started

Channel to actor: Player's finger has physical interaction with the screen to select their side

Secondary actors: System

Channel to secondary actors: System receives feedback to begin game

Open issues:

- First move advantage: How can we ensure an equal share of first moves after players choose their sides?

Use case: Check for Captures (Checkers)

iteration: 1

primary actor: System

Goal in context: The System checks for captures

Precondition: A move has been made

Trigger: A player moves

Scenario:

1. Players moves their piece (either jumps or does not)
2. system checks if a piece has been captured
3. System returns true if a capture occurred, otherwise false

Post conditions: The system accurately returns true or false based on the capture status of the move.

Exceptions:

1. System times out

Priority: High

When available: 1 iteration

Frequency of use: Frequent - Every time a player moves

Channel to actor: System receives feedback signalling a move has been made

Secondary actors: N/A

Channel to secondary actors: N/A

Open issues:

- Multiple captures: How should we check for multiple captures? Is it different from one capture?

Use Case: Validate Move

Iteration: 1

Primary Actor: System

Goal in Context: Ensure that a player's move is valid according to the rules of the game.

Preconditions:

1. The player is logged into their account.
2. The player is connected to an active game session.
3. It is a player's turn to make a move.
4. The player has selected a specific grid location on the game board.
5. The selected move complies with the basic rules of the game (e.g., selecting an empty cell in Tic-Tac-Toe or a valid jump in Checkers).

Trigger: A player attempts to place a piece or make a move on the board.

Scenario:

1. A player selects a location on the board to place a piece or move a piece.
2. The system verifies that it is the player's turn.
3. The system checks that the selected move follows the specific game rules:
    a. Tic-Tac-Toe:
        i. The system ensures the selected cell is unoccupied.
    b. Checkers:
        i. The system confirms that the move is within the piece's movement range (diagonal move for regular pieces, any direction for kings).
        ii. The system checks if the move is a valid capture, if applicable.
    c. Connect 4:
        i. The system verifies that the selected column has an open spot in the lowest available row.
4. If the move is valid, the system allows the move to proceed.
5. If the move is invalid, the system displays an error message and prompts the player to select a valid move.

Postconditions:

1. The move is validated and, if valid, will be executed on the board.
2. If the move is invalid, the system prevents the action and requests the player to make another selection.

Exceptions:

- A player attempts to place a piece outside the boundaries of the board.
- A player selects an occupied spot (e.g., in Tic-Tac-Toe).
- A player makes an invalid move for the selected piece (e.g., in Checkers, pieces can only move diagonally on dark squares).

Priority: High, as move validation is essential to prevent invalid actions in each game.

When Available: 1st iteration.

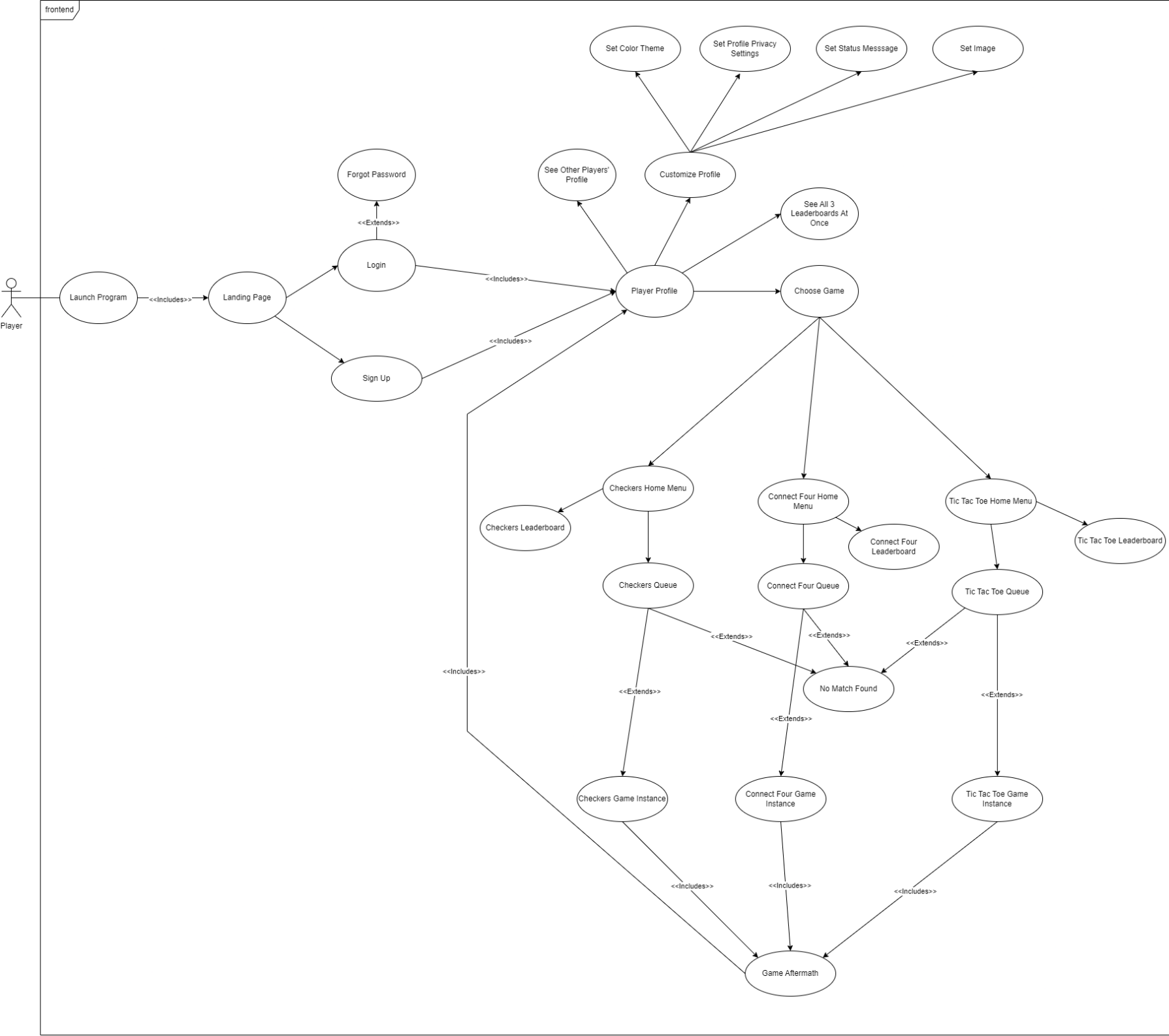
Frequency of Use: Multiple times per game, depending on game length.

Channel to Actor: The system automatically checks and notifies players via the game interface.

Secondary Actors: Both players in the game session.

Channels to Secondary Actors: If the move is valid, the updated board state is displayed on each player’s screen

Open Issues: None.


frontend
Player
Launch Program
<<Includes>>
Landing Page
Login
Forgot Password
<<Extends>>
Sign Up
<<Includes>>
<<Includes>>
Player Profile
See Other Players' Profile
Customize Profile
Set Color Theme
Set Profile Privacy Settings
Set Status Messsage
Set Image
See All 3 Leaderboards At Once
Choose Game
Checkers Home Menu
Checkers Leaderboard
Checkers Queue
Checkers Game Instance
Connect Four Home Menu
Connect Four Leaderboard
Connect Four Queue
Connect Four Game Instance
Tic Tac Toe Home Menu
Tic Tac Toe Leaderboard
Tic Tac Toe Queue
Tic Tac Toe Game Instance
<<Extends>>
<<Extends>>
<<Extends>>
<<Extends>>
<<Extends>>
<<Extends>>
No Match Found
<<Includes>>
<<Includes>>
<<Includes>>
<<Includes>>
Game Aftermath

Use Case: Launch Program

Iteration: 1

Primary Actor: User

Goal in Context: Executes the program for the user, opens up the framework for the user to access the profile. This use case will set up and allocate all necessary data and utilities to enable full functionality for the user. Necessary checks will be done to analyze all requirements before launching the program, otherwise will alert the user for any exceptions that may occur within the system.

Preconditions:

1. Users must have necessary files to be able to access the program, either by running the program through an integrated development environment, or through an executable.
2. Users must have an input medium such as a physical or virtual keyboard to be able to interact and enter necessary details.
3. Users must have an output medium such a monitor or screen to be able to see results created by the program.
4. Users must have a running machine, that means program requirements. **[Will be updated as the program is developed and published].**
5. Users must not change the contents of files, unless told otherwise by system prompts.
6. Users must make sure the machine is connected to the necessary connections, i.e. hosted network.

Trigger: Using an input medium the user launches the program either by running it through the integrated development environment, or through an executable.

Scenario:

1. Users download the necessary files through a verified shared domain platform (Gitlab or other sharing website).
2. Users read the README content to familiarize themselves with the program and its various functionality.
3. User ensures the current machine has met the specifications as laid out in the introduction documentation.

4. User launches the program either by running it through the integrated development environment at the program file location, or through an executable.
5. User successfully launches the program and is greeted by the launching page which will carry forward the user experience.

Post Conditions:

1. Users have allocated the necessary resources and connections to enable a full experience, from which they can browse games and see their profile.
2. User is greeted by the login/sign up page.
3. Users are able to close the program if chosen, and states are saved.

Exceptions:

1. User has not downloaded the necessary files to enable them full access to the program, and thus is unable to proceed. Program will terminate and deallocate any resources and connections hosted.
2. User has launched the program, but does not have the input device to continue further. Users may connect to an input device, or restart the program with an appropriate input device.
3. Users cannot execute the files because system requirements are not met, the program will not execute and alert users.
4. User has corrupt or missing files, the program will terminate or will fetch resources online when possible.

Priority: High, cannot access other features without being able to execute a program.

When available: First-second iteration

Frequency of Use: Once during the program lifecycle, unless requiring a restart.

Channel to Actor: Executable or through program files executed in an integrated development environment.

Secondary Actor: None.

Channel to Secondary Actor: N/A

Open Issues:

1. Verifying the files that the user has requires system privileges, which need to be considered carefully as to not impede on the user's privacy and security.
2. Program may require an update, and version mismatch can prevent the user from executing the program, and need to consider ways of devising version control.
3. Program may need logging of system prompts to track any issues that user may be experiencing, and potentially setting up an error code system.
4. If the correct files are not downloaded, the program may not execute to the initial state to alert users that files are corrupted, how can the system alert users without relying on the initial program startup.
5. New executing errors may arise, needing to enable a functionality so that users are able to (by permission) send error reports for code improvement.

Use Case: Landing Page

Iteration: 1

Primary Actor: User

Goal in Context: The initial start-up page, where the user is able to select whether to login or sign-up. Users are greeted by the landing page which includes the graphics elements and various details about the program, giving a snippet of the user experience. Users are able to browse through the program functions, however to proceed to the player profile, users must login or sign up with a new account.

Preconditions:

1. Users must have necessary files to be able to access the program, either by running the program through an integrated development environment, or through an executable.
2. Users must have an input medium such as a physical or virtual keyboard to be able to interact and enter necessary details.
3. Users must have an output medium such a monitor or screen to be able to see results created by the program.
4. Users must have a running machine, that means program requirements. **[Will be updated as the program is developed and published].**
5. Users must not change the contents of files, unless told otherwise by system prompts.
6. Users must make sure the machine is connected to the necessary connections, i.e. hosted network.

Trigger: Using an input medium the user launches the program either by running it through the integrated development environment, or through an executable.

Scenario:

1. Users download the necessary files through a verified shared domain platform (Gitlab or other sharing website).
2. Users read the README content to familiarize themselves with the program and its various functionality.
3. User ensures the current machine has met the specifications as laid out in the introduction documentation.

4. User launches the program either by running it through the integrated development environment at the program file location, or through an executable.
5. User successfully launches the program and is greeted by the landing page which will carry forward the user experience.
6. Users are able to view the program graphs and select whether they want to login to an existing account, or sign up using a new account.

Post Conditions:

1. User has successfully launched the program and has met all program requirements to be able to access all the futures available in the application.
2. Users are able to view graphic elements that showcase program functionality, and users can proceed by either signing up or logging into an existing account.
3. Users are able to view debugging and logged information that may include any errors that may have occurred during program launch.
4. Users are able to resize the program window after reaching the landing page to accommodate for any viewing screen.

Exceptions:

1. User has not met the pre conditions, land page resources have not been allocated and the user cannot interact with the program. Users must restart the program and ensure all files have downloaded as specified in the README file.
2. User has not resized the landing screen properly, some elements of the program are not displayed correctly. Users must resize the window to show elements properly and enable proper functioning.
3. An unexpected error can arise even with correct files (may due to errors in production code), so users can check logged files for error diagnosis.

Priority: High, cannot access other features without being able to execute a program.

When available: First-second iteration

Frequency of Use: Multiple times when a program is launched, users can return to the landing page at any given moment.

Channel to Actor: Executable or through program files executed in an integrated development environment.

Secondary Actor: None.

Channel to Secondary Actor: N/A

Open Issues:

1. Need to address a consistent formal theme that is strictly followed throughout the application, otherwise users can be misguided when expecting similarity across applications.
2. Need to make applications compatible with screen sizes to ensure a consistent experience, however layout may need to be adapted and changed where applicable.
3. Need to add an timeout-session where if the program does not receive a response the program is terminated to avoid application from overusing resources when stuck due to uncaught errors (May show debugging information automatically after a given duration).

Use Case: Sign Up

Iteration: 1

Primary Actor: User

Goal in Context: Allows the user to create a profile that enables customer customization, save states, and personalized experience within the program. Sign up will require the user to enter the following: name, date of birth, location, username, and password (required twice for confirmation). Which will be saved to the server to be saved before the user is able to login. Allows the user to begin customizing their experience and save their progress that can be viewed by logging in.

Preconditions:

1. User has entered the correct data type and details into the text fields as required by the program, and no unrecognized characters are used.
2. User has chosen an username that has not been selected before, and enters the matching password in both of the text fields.
3. User is enters information that is factual and representative of the user
4. User does not currently have an account registered before and is not signing up on the behalf of another individual.
5. Users must be able to enter the matching credentials when using the account to login into the system once the account has been created.
6. User has an appropriate input device to enter the relevant details into the text fields, and has launched the program without any missing files [see use case]

Trigger: User selects the sign up option from the landing page after executing the program, and enters the required fields correctly.

Scenario:

1. User executes the program using the integrated development environment or through an executable.
2. User selects the sign up option from the landing page
3. User enters the following information into the required text fields
    a. Name
    b. Date of birth
    c. Location

    d. Username
    e. Password
4. User correctly enters the information and meets all requirements, user submits the form
5. Confirmation is presented to the user outlining the username and the necessary details filled out prior.
6. User is proceeds to the player profile which has null/empty fields, and needs to play to collect data.

Post Conditions:

1. User has created an unique account in the database, which they can use to access/personalize other functions within the program
2. Users are able to logout at any moment and all data previously collected/updated will be stored onto a database, and can retrieve data by logging back into the program.
3. User is able change details of their account once the account has been created through player profile
4. Users are able to see how their profile stacks up against other players, note users must create an account before they are able to compare stats in the leaderboard.

Exceptions:

1. User has entered an username that already exists, user can either login to their existing account (through forget password) or create another account within the domain.
2. User has entered the required information incorrectly into the text fields, user is prompted with an error noting the details that must be retyped.
3. User has inputted a password that does not match with the previous password, user must have matching passwords in both of the text fields before being able to create an account.
4. User has entered unrecognizable characters/symbols that cannot be validated properly, must check if the characters are a part of the selected subsection.

Priority: High, new users must have an account to track and use any program functions.

When available: First-second iteration

Frequency of Use: Once when freshly installing the program, unless creating another account.

Channel to Actor: Program landing page.

Secondary Actor: None.

Channel to Secondary Actor: N/A

Open Issues:

1. The pool of usable usernames may be too limiting, how can the number of unique usernames available to new users be increased, may consider adding username requirements.
2. Inorder to provide sufficient privacy and security to users, may consider adding password requirements which will enable users to be considerate when choosing passwords.
3. When the username already exists, it may direct the user to the forget-password to recover the password, however it may require the user to enter a matching name to correctly identify the user has previously created an account.
4. May require the use of security questions to be added to the sign up form to ensure that users are authenticated when attempting to retrieve their account through forget-password.

Use Case: Login

Iteration: 1

Primary Actor: User

Goal in Context: Allows the user to create a profile that enables customer customization, save states, and personalized experience within the program. Login will take a username and password, which will be authenticated on a server and verified, before allowing the user to login. Enables quick access to game library and leaderboard stats that otherwise would not be saved/accessible.

Preconditions:

1. Users must have previously created a unique username and password before attempting to login.
2. User attempts to login with a valid username and password which they are authenticated to/own.
3. User has correctly entered the unique username and password that pertains to them.
4. If a user is unable to login into their existing account, they utilize the forget-password function [see use case].
5. Username and password is entered using an input device that enables full compatibility.

Trigger: User enters an unique username and password into the required text fields before proceeding to the landing page.

Scenario:

1. User executes the program using the integrated development environment or through an executable.
2. User selects login option from the landing page
3. User enters their unique username and password in the correct text fields, user proceeds to login
4. User is authenticated and proceeds to the player profile.

Post Conditions:

1. User is logged into their selected player profile
2. User is able to personalize their experience through the program tools
3. Users are able access the game library and save their progress/stats.
4. Users are able to see the leaderboards and their rankings, which are tied to their unique username and saved states.

Exceptions:

1. User is not able to login into their account, and must be redirected to the forget-password page [see use case]
2. User does not fill text fields for username and password, and/or uses incompatible characters, the program will prompt the user to retry.
3. Network may be timed out through multiple attempts at password verification, the user must restart the program and retry.

Priority: High, cannot access other features without logging in.

When available: First-second iteration

Frequency of Use: Once during the program lifecycle, unless requiring a restart.

Channel to Actor: Program landing page.

Secondary Actor: None.

Channel to Secondary Actor: N/A

Open Issues:

1. Need to address password attempting in a way that limits unauthorized access to the system, while allowing enough attempts for user convenience
2. Allow the password-forget option to appear when the username is valid, otherwise the user cannot tell if the username is correct.
3. Before the username and the password are authenticated, must make sure that the text fields are filled in to let the user know the mistakes in their entry.

Use Case: Forget Password

Iteration: 1

Primary Actor: User

Goal in Context: Allow users that have forgotten their passwords to reset their account password. Users must be able to prove they own and are authenticated into the account, by entering details that were used when signing up. This can include but not limited to: Name, location of account creation, security questions, and the pertaining username of the account.

Preconditions:

1. User must have an account in the database to recover
2. User must be authenticated to recover the password, be the original owner and have full access to their data
3. Users must have relevant details which are needed to authenticate if the owner of the account is the individual attempting to recover the password.
4. Reset password meets the criteria, and is valid to be renewed for the account that is being recovered.
5. All details of the account must not change during password recovery, which may create conflicts when verifying data.

Trigger:

1. **Signing Up**: User has entered a name and username that already exists on the database, user is redirected to forget-password to recover their account.
2. **Login:** Users cannot authenticate into their account and must recover their account by correctly entering partial details.

Scenario:

1. User executes the program using the integrated development environment or through an executable.
2. User selects login option from the landing page
    a. User cannot login to their account and enters the forget-password option
    b. User enters the name, username, and necessary details to recovery account (note based on account familiarity)
3. User selects signup and enters an username that exists on the database

    a. User enters the name, username, and necessary details to recovery account (note based on account familiarity)
4. User receives confirmation, and is able to login to the system using their renewed credentials
5. User selects login option and enters necessary details (see use case)
6. User proceeds to the player profile

Post Conditions:

1. The valid password that is entered into the text field is the new allocated password for the given account.
2. Users are able to use the new password to login to the system, and all instances have been replaced.
3. Users have recovered their previous save states, including profile customization and leaderboards stats.
4. Users are able to experience all program functions that ordinary new/existing users have access to.

Exceptions:

1. User has not been authenticated to change password, the program returns the user back to the landing page, and password recovery attempts on the account will be noted down. Users who have logged into the account will have a system prompt to show the unrecognized system recovery.
2. New password does not meet the requirements set by the system, the program will alert the user that the password does not meet criteria and will need to retry.
3. Password recovery attempt on the account has exceeded the set limit, the user will be logged out and will need to continue by creating a new account.
4. User abandons password recovery while filling out required text field, program will redirect user back to the landing page, where user can either choose to login or sign up (see use cases)

Priority: High, cannot access other features without logging in.

When available: First-second iteration

Frequency of Use: Once during the program lifecycle, unless requiring a restart.

Channel to Actor: Program landing page.

Secondary Actor: Backend/network Request

Channel to Secondary Actor: In special cases, where a user cannot be authenticated into the system, a backend/network request can be sent to inspect details and manually verify the user.

Open Issues:

1. Currently login does not accommodate for utilities that are used by forgot-password (i.e. security questions), which may need to be revised after the initial iteration to allow for complex user interactions.
2. User verification must be tested thoroughly to allow access only to account owners, and not any unauthorized individuals which are attempting to login fraudulently.
3. Need to set a limit of password changes to ensure user security is not compromised when changing password, at some limit users may need to create a new account.
4. Security questions which will be added after the first iteration need to be representative of the user, in a way that uniqueness allows for secure encryption.
5. Users may forget their passwords overtime, need to devise a way to reset passwords after a certain duration or routinely after the user has finished a game session.

Descriptions for User profile, view the leaderboard, view other players' profiles, and choose game.

Use cases: User Profile (Main)

Iteration: 1

Primary Actor: Player

Goal in Context: A main page for a player to view its own profile and access to other features (view leaderboards/other players' profiles, choose game etc)

Preconditions: Player either signed in or logged in.

Trigger: Player is successively logged in.

Scenario:

1. Player is successfully logged in.
2. Player profile page pops up. This page includes the game nickname, profile picture, name and user's game stat.
3. Bottom of the page has buttons to click; switch user, view leader board, view my stats, view other player's profiles, customize own profile, choose game and etc.
4. Player will be moved to a different page according to the button clicked.
5. (Switch User: goes back to the login page)
6. (Customize Profile: Covered by the use case description: "Profile Customization")

Post Conditions: According to the player's decision(button), the player will be led to a new page(window).

Exceptions:

1. The player's game stats are not fully retrieved
2. Incorrect stats are retrieved
3. Buttons don't work. The player can't move to a different page or is moved to the wrong page.

Priority: High; This page will always be accessed. Works as the main controller.

When Available: Within 1st sprint.

Frequency of Use: High**;** Whenever a player decides to use this program(app)

Channel to Actor: User interfaces (GUI, screen; monitor, keyboard, speaker, mouse, etc.)

Secondary Actors: Customer service

Channel to Secondary Actors: Email, phone, (service request page)

Open Issues:

1. Should there be a "setting" page where the player can change game settings(color theme, font, sound, notification, etc.)?

Use cases: View Leaderboard

Iteration: 1

Primary Actor: Player

Goal in Context: Allow player to see the overall leaderboard (for all three games; Tic Tac Toe, Connect Four, and Checkers)

Preconditions:

1. Player is on the main profile page.
2. Stats are retrieved from the “aftermath” and “leaderboard” for each game.

Trigger: Player clicked the button “view leaderboard” from the main profile page.

Scenario:

1. Player is on the main profile page
2. Player clicks the button “view leaderboard”
3. Player is now moved to a different page that shows the leaderboard of all three games.

Post Conditions: After checking the leaderboard player can come back to the main profile page by clicking the “Back to main” button.

Exceptions:

1. Some of the game stats are not retrieved.
2. Incorrect game stats are retrieved

Priority: Low; This page will never be accessed unless the player wants to view the leaderboard.

When Available: within 1st sprint

Frequency of Use: Low; zero or once per player

Channel to Actor: User interfaces (GUI, screen; monitor, keyboard, speaker, mouse, etc.)

Secondary Actors: customer service

Channel to Secondary Actors: email, phone, (service request page)

Open Issues: N/A

Use cases: View other player's profile

Iteration: 1

Primary Actor: Player

Goal in Context: Allow player to view other players' profile.

Preconditions:

1. Player was able to log in and is on the main profile page.
2. Stats of other players are retrieved from "aftermath".

Trigger: Player clicks the "View other players' profile" button.

Scenario:

1. Player is on the main user profile page.
2. Player clicks the "View other players' profile" button.
3. Player is now on a different page that has a search bar to look up other players.
4. Player types in other players' names that he/she wants to view their profile.
5. The profiles and stats for the searched players are shown below the search bar.
6. Player can go back to the main profile page by clicking "Back to main" button.

Post Conditions: Player is back to the main user profile page

Exceptions:

1. Player information is not retrieved fully
2. Incorrect player information is displayed.
3. The buttons don't work

Priority: Medium

When Available: within 1st iteration

Frequency of Use: low

Channel to Actor: User interfaces (GUI, screen; monitor, keyboard, speaker, mouse, etc.)

Secondary Actors: customer service

Channel to Secondary Actors: email, phone, (service request page)

Open Issues:

1. How many players can be shown on one page? Should it be just one or several?
2. Is the player allowed to look up more than one player by typing their names in the search box all at once?
   -example: [Christiano Ronaldo, David Beckham…]

Use cases: Choose Game

Iteration: 1

Primary Actor: Player

Goal in Context: Allow the player to choose a game to play.

Preconditions: Player is successfully logged in and on the main user profile page.

Trigger: Player clicks the “choose game” button on the main user profile page.

Scenario:

1. Player is on the main user profile page.
2. Player clicks the “choose game” button
3. Player is now on the choose game page. This page has three buttons(one for each game) with a corresponding picture and a short introduction(preview) below.
4. Player decides what game to play and clicks the corresponding button.
5. Player is now on a new page where the game starts.
6. Player can go back to the main profile page by clicking the “Back to main” button.

Post Conditions: Player is either back on main menu page or has started playing a game.

Exceptions:

1. The button doesn’t work
2. The button is connected to a different game (Connect 4 button connected to Checkers)

Priority: Essential. Must be implemented.

When Available: Within 1st iteration

Frequency of Use: Medium**;** if the player plays one game, this page won’t be used much.

Channel to Actor: User interfaces (GUI, screen; monitor, keyboard, speaker, mouse, etc.)

Secondary Actors: Customer service

Channel to Secondary Actors: Email, phone, (service request page)

Open Issues:

1. Can the short introduction of the games be shown as a pop-up by hovering the cursor over the buttons?

Use Case: Profile Customization

Iteration: 1.

Primary Actor: Player.

Goal in Context: Enables the player to personalize their profile with custom elements such as avatars, color themes, and personal status messages.

Preconditions: The player has an active account and is logged in to the platform.

Trigger: The player selects the option to customize their profile from the About Player profile screen.

Scenario:

1) Player Accesses Profile Customization Page
   a) The player navigates to the profile page and selects "Customize Profile," opening the customization interface.

2) Customization Options Displayed
   a) The profile customization page displays a variety of options, for example:
      i) Avatar Selection: A set of avatars (default and/or unlocked) for the player to choose from.
      ii) Color Themes: Options to change the color theme of their profile background and UI elements.
      iii) Status Message: A field to add or update a personal status message visible to other players.
      iv) Privacy Settings: Controls to adjust the visibility of specific profile elements (ex: stats, status message) for friends or the public.

3) Apply and Save Changes
   a) Once satisfied with the selections, the player can apply and save their changes.
   b) A confirmation message is displayed indicating successful profile updates.

4) Changes Reflect on Profile
   a) Once saved. The profile updates immediately, allowing the player and others to see the new avatar, theme, status message etc.

Post-Conditions: The player is returned to the lobby/player profile screen, and all game statistics and progress are accurately updated in the system.

Exceptions:

1. Invalid or Restricted Content in Status Message.
   a. If the player enters inappropriate language or restricted content in their status message, an error message is displayed explaining that the status message contains prohibited content.

Priority: High.

When available: First Iteration.

Frequency of Use: Frequent. User may wish to change their profile at any time.

Channel to Actor: Player interacts with the game interface via a graphical user interface.

Secondary Actor: N/A.

Channel to Secondary Actor: N/A.

Open Issues:

1. Determine if stats can be accessed later in the player's profile for review after leaving the aftermath screen.
2. Clarify whether players can manually choose to skip or delay the stats display and proceed directly to the lobby/player profile.

Use Case: Customize Color Theme

Primary Actor: Player.

Goal in Context: Allow the player to personalize their profile with a custom color scheme.

Preconditions: The player is logged into their account and has access to the profile customization page.

Trigger: The player selects the option to customize the profile color theme.

Scenario:

1. The player selects the color theme customization option.
2. A range of color themes is displayed, including default and unlocked options.
3. The player chooses a color theme for their profile.
4. The selected color theme is applied and saved, updating the player's profile immediately.

Post-Conditions: The new color theme is visible on the profile.

Exceptions:

1. If the color theme options fail to load, an error message redirects the player to
2. the customization page.
3. If the chosen theme cannot be saved, an error message prompts the player to try
4. again.

Priority: Medium.

When Available: First Iteration.

Frequency of Use: Frequent. Whenever the player wishes to update their color theme.

Channel to Actor: Player interacts with the color theme selection via the GUI.

Secondary Actor: N/A.

Channel to Secondary Actor: N/A.

Open Issues:

1. Will some themes be available only through achievements or purchases?

Use Case: Profile Image Selection

Iteration: 1.

Primary Actor: Player.

Goal in Context: Enable the player to personalize their profile by selecting a profile image from a predefined set or by uploading an image of their choice.

Preconditions: The player has an active account and is logged into the platform and the player has accessed the Profile Customization page.

Trigger: The player selects the option to change their profile image within the Profile Customization interface.

Scenario:

1) **Player Accesses Profile Image Selection**
   a) The player selects the "Profile Image" option within the Profile Customization page, opening a window with image options.
2) **Predefined Profile Image Options Displayed**
   a) The system displays a range of default profile images for the player to choose from.
3) **Option to Upload Custom Profile Image**
   a) The player is presented with an option to upload their own image. Upon selection the player is prompted to choose an image file from their device.
4) **Apply and Save Profile Image**
   a) The player confirms their choice by selecting "Apply."
   b) A confirmation message appears, indicating that the profile image has been successfully updated.
5) **Image Reflects on Profile**
   a) The new profile image is updated on the player's profile.

Post-Conditions: The selected or uploaded profile image is saved and displayed on the player's profile. If successful, the player is redirected to the Profile Customization page to continue customizing or exit.

Exceptions:

1) **Invalid or Restricted File Type for Upload**
   a) The player attempts to upload an image file that is not in a supported format.
2) **File Size Exceeds Limit**
   a) The player attempts to upload an image that exceeds the file size limit.

Priority: High.

When Available: First Iteration.

Frequency of Use: Frequent. Players may update their profile image at any time.

Channel to Actor: Player interacts with the profile image selection interface via the GUI.

Secondary Actor: N/A.

Channel to Secondary Actor: N/A.

Open Issues: None.

Use Case: Set Profile Privacy Settings

Primary Actor: Player

Goal in Context: Allow the player to adjust privacy settings for profile elements (e.g., stats, status message, achievements) to control visibility.

Preconditions: The player is logged into their account and has access to the profile customization page.

Trigger: The player selects the privacy settings option.

Scenario:

1. The player clicks on the privacy settings option.
2. Options to adjust visibility for profile elements (ex: stats, status message) are
3. displayed.
4. The player chooses visibility settings (ex: friends only, public, or private) and
5. saves the preferences.
6. The new privacy settings are applied.

Post-Conditions: The updated privacy settings are saved and enforced on the player's profile.

Exceptions:

1. If privacy settings fail to save, an error prompts the player to try again.

Priority: High.

When Available: First Iteration.

Frequency of Use: Frequent. Whenever the player wishes to update their visibility preferences.

Channel to Actor: Player interacts with privacy settings via the GUI.

Secondary Actor: N/A.

Channel to Secondary Actor: N/A.

Open Issues:

1. What is the default privacy settings for new users?

Use Case: Set Status Message

Primary Actor: Player.

Goal in Context: Allow the player to add or update a personal status message on their profile.

Preconditions: The player is logged into their account and has access to the profile customization page.

Trigger: The player selects the option to edit their status message.

Scenario:

1. The player clicks the status message option.
2. A text input field appears for the player to type their status message.
3. The player enters their status message.
4. Player saves the message.
5. The status message updates on the profile and is visible based on privacy settings.

Post-Conditions: The new status message appears on the player's profile page.

Exceptions:

1. If the status message cannot be saved, an error prompts the player to try again.

Priority: High.

When Available: First Iteration.

Frequency of Use: Frequent. Whenever the player wishes to update their status message.

Channel to Actor: Player enters the status message via the GUI.

Secondary Actor: N/A.

Channel to Secondary Actor: N/A.

Open Issues: None.

Use Case Descriptions for: *(1) Connect 4, (2) Tic Tac Toe, (3) Checkers*

Use Case: Connect 4 Home Menu

Iteration: 1

Primary Actor: Player

Goal In Context: Allow the player to access the main Connect 4 game interface and select options such as starting a new game, viewing the leaderboard, or joining a queue for matchmaking.

Preconditions:

- The player is logged in and has access to their profile.
- The Connect 4 game module is available.

Trigger: The player navigates to the Connect 4 game section from the Player Profile Home Menu.

Scenario:

1. The player selects “Connect 4” from the Player Profile Home Menu.
2. The system loads the Connect 4 Home Menu.
3. The player sees options to view the Connect 4 leaderboard, or join the matchmaking queue.
4. The player selects their desired option (e.g., “Join Queue”).
5. The system directs the player to the chosen option.

Post Conditions: The player has accessed their chosen option within the Connect 4 module.

Exceptions:

- **E1:** The Connect 4 module is unavailable. The system shows an error message and redirects the player to the Player Profile Home Menu.

Priority: High

When Available: Upon player login and successful navigation to Connect 4.

Frequency of Use: Frequent, as this is the entry point for Connect 4-related activities.

Channel to Actor: User Interface (UI)

Secondary Actors: None

Channels to Secondary Actors: N/A

Open Issues:

- Should the Connect 4 Home Menu include additional options, such as game settings or tutorials? Or play with AI? Or local multiplayer?

Use Case: Connect 4 Game Queue

Iteration: 1

Primary Actor: Player

Goal In Context: To allow the player to enter a queue for matchmaking in order to find an opponent for a Connect 4 game.

Preconditions:

- The player has accessed the Connect 4 Home Menu.
- The matchmaking service is active.

Trigger: The player selects the option to join the Connect 4 game queue.

Scenario:

1. The player selects “Join Queue” from the Connect 4 Home Menu.
2. The system adds the player to the Connect 4 game queue.
3. The system continuously searches for a compatible opponent.
4. If an opponent is found, the system notifies the player and initiates the Connect 4 Game Instance.

Post Conditions: The player has either joined a match or is informed that no match is currently available.

Exceptions:

- **E1:** No players are available in the queue. The system displays a “No Match Found” message.
- **E2:** Matchmaking service is down. The system shows an error message and returns the player to the Connect 4 Home Menu.

Priority: High

When Available: Anytime the player chooses to enter the queue.

Frequency of Use: Moderate, dependent on player activity and matchmaking wait times.

Channel to Actor: UI notification (possibly with a “Searching for opponent…” indicator)

Secondary Actors: Matchmaking service

Channels to Secondary Actors: System-to-system communication with matchmaking service

Open Issues:

- What criteria are used to find a compatible opponent (e.g., skill level)?
- Should there be an estimated wait time shown to the player?

Use Case: Connect 4 Game Instance

Iteration: 1

Primary Actor: Player

Goal In Context: Allow the player to participate in a Connect 4 game against an opponent.

Preconditions: A match has been successfully created, and an opponent has been assigned.

Trigger: The system initiates the game instance upon finding an opponent for the player in the queue.

Scenario:

1. The system matches the player with an opponent and starts the game instance.
2. The player is notified that the game is starting and is presented with the Connect 4 game board.
3. The player and the opponent take turns making moves until the game ends (either by winning, losing, or a draw).
4. The system records the game result and updates the leaderboard.
5. The player is shown the game result screen (victory, defeat, or draw).

6. The player has the option to return to the Connect 4 Home Menu or view the leaderboard.

Post Conditions: The game results are recorded, and the player is directed to the game aftermath options.

Exceptions:

- **E1:** Opponent disconnects. The player is informed, and the game ends prematurely.
- **E2:** Connection loss. The system saves the game state temporarily or informs the player of the disconnection.

Priority: High

When Available: Once matchmaking is complete.

Frequency of Use: Dependent on player's success in finding a match.

Channel to Actor: UI with real-time updates

Secondary Actors: Opponent player

Channels to Secondary Actors: Network communication for real-time game moves

Open Issues:

- Should there be an option to rematch with the same opponent?
- How is the game state handled if a player disconnects?

Use Case: No Match Found

Iteration: 1

Primary Actor: Player

Goal In Context: Notify the player that no match is currently available in the queue and provide options to retry or exit.

Preconditions: The player has joined the Connect 4 Game Queue.

Trigger: The system fails to find an opponent within a defined timeout period or due to low player availability.

Scenario:

1. The player joins the Connect 4 Game Queue.
2. The system continuously searches for an opponent.
3. After a specified timeout period, the system fails to find a match.
4. The system displays a “No Match Found” notification to the player.
5. The player is given options to retry joining the queue or to return to the Connect 4 Home Menu.

Post Conditions: The player is informed that no match was found and can choose to retry or exit.

Exceptions:

- **E1:** System experiences an error in retrieving player queue status. An error message is shown, and the player is redirected to the Connect 4 Home Menu.

Priority: Medium

When Available: During periods of low player activity or when the matchmaking queue has a timeout.

Frequency of Use: Occasional, depending on player availability and queue demand.

Channel to Actor: UI notification

Secondary Actors: None

Channels to Secondary Actors: N/A

Open Issues:

- How long should the system wait before displaying the “No Match Found” message?
- Should the player receive an estimated wait time or indication of current queue length?

Use Case: Connect Four Leaderboard

Iteration: 1

Primary Actor: Player

Goal In Context: To allow the player to view the leaderboard for Connect Four, displaying essential rankings and game-related statistics of top players.

Preconditions:

- The player has accessed the Connect Four Home Menu.
- Leaderboard data is available and up-to-date.

Trigger: The player selects the “Leaderboard” option from the Connect Four Home Menu.

Scenario:

1. The player navigates to the Connect Four Home Menu.
2. The player selects “Leaderboard.”
3. The system retrieves the latest leaderboard data for Connect Four.
4. The system displays the leaderboard, showing the following statistics for each player:
    - **Rank**: The player’s position based on win rate or score.
    - **Username**: The display name of each ranked player.
    - **Total Wins**: The total number of games the player has won.
    - **Win Rate**: The percentage of games won out of total games played.
    - **Longest Win Streak**: The highest number of consecutive wins achieved.
    - **Games Played**: The total number of games the player has participated in.
    - **Last Played Date**: The date of the player’s most recent game.
    - **Moves to Win (Average)**: The average number of moves the player needed to win a game.
5. The player can scroll through or search the leaderboard to find specific players or rankings.
6. The player can filter the leaderboard based on categories like “Top Wins” or “Highest Win Rate.”
7. The player can return to the Connect Four Home Menu or view their own profile from the leaderboard page.

Post Conditions: The player has successfully viewed the Connect Four leaderboard and can navigate back to other Connect Four options.

Exceptions:

- **E1:** The leaderboard data fails to load due to a network issue. The system displays an error message and provides an option to retry or return to the Connect Four Home Menu.

- **E2:** The leaderboard service is temporarily unavailable. The system informs the player and directs them to try again later or return to the Connect Four Home Menu.

Priority: Medium

When Available: Anytime the player is in the Connect Four Home Menu.

Frequency of Use: Occasional, based on the player's interest in checking rankings and scores.

Channel to Actor: UI display on the leaderboard screen

Secondary Actors: Leaderboard database/service

Channels to Secondary Actors: System-to-database communication to retrieve leaderboard information

Open Issues:

- Should the leaderboard allow additional filtering options (e.g., friends, region)?
- How frequently should the leaderboard data be refreshed for accuracy?

Use Case: Tic Tac Toe Home Menu

Iteration: 1

Primary Actor: Player

Goal In Context: To enable the player to access the main game interface for Tic Tac Toe and select options such as starting a new game, viewing the leaderboard, or joining the matchmaking queue.

Preconditions:

- The player is logged in and has access to their profile.
- The Tic Tac Toe module is active.

Trigger: The player navigates to the Tic Tac Toe game section from the Player Profile Home Menu.

Scenario:

1. The player clicks “Tic Tac Toe” from the Player Profile Home Menu.
2. The system loads Tic Tac Toe Home Menu.
3. Player is given options to either view the Tic Tac Toe leaderboard or join the matchmaking queue.
4. The player selects their desired option (e.g., “Join Queue”).
5. The system directs the player to the chosen option.

Post Conditions: The player is successfully able to access their chosen feature in the Tic Tac module.

Exceptions:

- **E1:** The Tic Tac Toe module is unavailable. The system shows an error message and redirects the player to the Player Profile Home Menu.

Priority: High

When Available: When player is logged in & navigates to Tic Tac Toe

Frequency of Use: Frequent, as this is the starting point for Tic Tac Toe-related activities.

Channel to Actor: User Interface (UI)

Secondary Actors: None

Channels to Secondary Actors: N/A

Open Issues: N/A

Use Case: Tic Tac Toe Game Queue

Iteration: 1

Primary Actor: Player

Goal In Context: To allow the player to enter a matchmaking queue to find an opponent for the Tic Tac Toe game.

Preconditions:

- The player has accessed the Tic Tac Toe Home Menu.
- Matching making feature is working.

Trigger: The player selects the option to join the Tic Tac Toe game queue.

Scenario:

1. The player selects “Join Queue” from the Tic Tac Toe Home Menu.
2. The system adds the player to the Tic Tac Toe game queue.
3. The system searches for a opponent.
4. Once opponent is found, system notifies the player and initiates the Tic Tac Toe Game Instance.

Post Conditions: Player either joins a match or gets a message that a match is not currently available.

Exceptions:

- **E1:** No players are available in the queue, so the system displays a “No Match Found” message.
- **E2:** Matchmaking service is offline or is malfunctioning so the system shows an error message and returns the player to the Tic Tac Toe Home Menu.

Priority: High

When Available: When the player clicks to enter a queue.

Frequency of Use: Moderate, depending on matchmaking demand for users

Channel to Actor: UI notification

Secondary Actors: Matchmaking service

Channels to Secondary Actors: System-to-system communication with matchmaking service

Open Issues:

- Should players be matched based on certain skill level or expertise?
- Extension to above: If there isn’t a player that can match skill levels, can the user be notified if & when a match is found?

Use Case: Tic Tac Toe Game Instance

Iteration: 1

Primary Actor: Player

Goal In Context: Enable the player to play a Tic Tac Toe game against another player.

Preconditions: A match has been successfully set up and an opponent is found from queue.

Trigger: The system begins the game instance when a suitable opponent is

Scenario:

1. System pairs player with it's opponent and initiates the game instance.
2. Player then receives a notification that the game is starting and is presented with the Tic Tac Toe game board
3. Then, the player and their opponent take turns making moves until the game ends (either a win, loss or draw)
4. The system records outcome of the game and updates the leaderboard with the respective winner.
5. The player is then directed to the result screen which displays "victory", "defeat" or "draw".
6. Player can choose to return to the Tic Tac Toe home screen or view the leaderboard from the results page.

Post Conditions: Game results are then recorded, and the player is directed to the game aftermath options.

Exceptions:

- **E1:** If the opponent disconnects or withdraws from game, player is notified and the game ends
- **E2:** If there is a connection issue on the player's end, system will save & freeze the game

Priority: High

When Available: Once matchmaking is done.

Frequency of Use: Varies, based on player's success in finding an opponent.

Channel to Actor: UI

Secondary Actors: Opponent user

Channels to Secondary Actors: Network communication for Tic Tac Toe moves

Open Issues:

- Should players be able to send challenges or game requests to recent opponents or opponents who are higher up on leaderboard?

Use Case: No Match Found

Iteration: 1

Primary Actor: Player

Goal In Context: Tells the player that the system was unable to find a match for them & it then provides options for them to exit the game or retry.

Preconditions: The player has joined the Tic Tac Toe Game Queue.

Trigger: If the system is unable to find an opponent for the player

Scenario:

1. The player joins the Tic Tac Toe Game Queue.
2. System searches for an opponent
3. After some time (5 minutes for ex), the system fails to find a match.
4. The system displays a “No Match Found” pop-up to the player
5. User is then given options to try joining the queue again or return to the Tic Tac Toe Home Menu.

Post Conditions: No match was found and player can choose to exit, retry later or try again now

Exceptions:

- **E1:** If there is an error getting the player’s queue number, error message is shown & player is then redirected to Tic Tac Toe home menu

Priority: Medium

When Available: If the matchmaking queue times out or if there aren’t enough players in the queue

Frequency of Use: Occasional

Channel to Actor: UI notification

Secondary Actors: None

Channels to Secondary Actors: N/A

Open Issues:

- Would implementing a “waiting room” feature with instructions for the game or mini challenges be a good way to increase user retention?

Use Case: Tic Tac Toe Leaderboard

Iteration: 1

Primary Actor: Player

Goal In Context: enables the player to view leaderboard for Tic Tac Toe & shows rankings in order of top player scores

Preconditions:

- The player passed the Tic Tac Toe Home Menu.

Trigger: The player selects the “Leaderboard” option from the Tic Tac Toe Home Menu.

Scenario:

1. The player navigates to the Tic Tac Toe Home Menu.
2. The player selects “Leaderboard.”
3. The system retrieves the latest leaderboard data for Tic Tac Toe.
4. The system displays the leaderboard, showing the following statistics for each player:
    - **Rank**: position based on win rate/score.
    - **Username**: display name of ranked players.
    - **Total Wins**
    - **Win Rate**: percentage of games won over total games.
    - **Longest Win Streak**: longest number of wins in a row.
    - **Games Played**: total number of games the player has done.
    - **Last Played Date**: date of most recent game.
    - **Moves to Win (Average)**: average number of moves player needs to take to win.
5. The player can scroll through or search the leaderboard to find specific players or rankings.
6. The player can filter the leaderboard (longest streak, most games played etc.)
7. The player can return to the Tic Tac Toe Home Menu

Post Conditions: The player has successfully viewed the Tic Tac Toe leaderboard and can navigate back to other Tic Tac Toe options.

Exceptions:

- **E1:** if the leaderboard data fails to load due to a network issue, the system will display an error message and provide the option to retry or return to the Tic Tac Toe Home Menu.
- **E2:** if the leaderboard is down, then the system informs the player and directs back to the Tic Tac Toe Home Menu.

Priority: Medium

When Available: Anytime the player is at the Tic Tac Toe Home Menu.

Frequency of Use: Occasional

Channel to Actor: UI display

Secondary Actors: Leaderboard database/service

Channels to Secondary Actors: System-to-database communication to retrieve leaderboard information

Open Issues:

- Would adding achievements, badges or premium features increase player retention and engagement?

Use Case: Checkers Home Menu

Iteration: 1

Primary Actor: Player

Goal In Context: Player is at the Home Menu screen for Checkers, where they can choose to enter the queue to play the game or view the leaderboard.

Preconditions:

- The player has logged in and is at the Player Profile Home Menu.
- The Checkers game is available.

Trigger: The player chooses to play Checkers and clicks on Checkers.

Scenario:

1. The player has logged into their account.
2. They are now at the Player Profile Home Menu.
3. They choose to play Checkers and click onto the game.
4. They are brought to the Home Menu screen for Checkers.
5. They can now choose to view the leaderboard or join a queue.

Post Conditions: The player is directed to view the leaderboard for Checkers, or to join the queue for the game.

Exceptions:

- **E1:** The Checkers module is unavailable, system shows an error message and the player is redirected to the Player Profile Home Menu.

Priority: High

When Available: After the player logs in and arrives at the Checker Home Menu.

Frequency of Use: Frequent, shown every time the player wants to play Checkers.

Channel to Actor: UI

Secondary actors: None

Channels to secondary actors: N/A

Open Issues:

- Should there be more options for the game at the Home Menu for the player to choose from?

Use Case: Checkers game queue

Iteration: 1

Primary Actor: Player

Goal In Context: The player would be able to join a queue to get matched up with an opponent.

Preconditions:

- The player has logged in and is at the Checkers Home Menu.
- The matchmaking service is available.

Trigger: The player chooses “Join Queue” from the Checkers Home Menu.

Scenario:

1. The player has logged into their account.
2. They choose to play Checkers and selects “Join Queue”.
3. The matchmaking system searches for an opponent.
4. If they are matched up with another player, they are directed to the Checkers game instance.

Post Conditions: The player is successfully matched up with another player, or they could not be matched up because no match is found.

Exceptions:

- **E1:** No other player wants to play Checkers at the same time and matching up fails.
- **E2:** The matchmaking system is down, the system shows an error message and redirects the player to the Checkers Home Menu.

Priority: High

When Available: When a player chooses to join a queue.

Frequency of Use: Moderate, depends on the player’s choices and wait times.

Channel to Actor: UI

Secondary actors: Matchmaking service

Channels to secondary actors: System-to-system communication

Open Issues:

- Should the player be able to choose to cancel queuing and get back to the game Home Menu?

Use Case: Game instance

Iteration: 1

Primary Actor: Player

Goal In Context: The player can play the Checkers game with an opponent.

Preconditions: The player has logged in and is matched up successfully with another player.

Trigger: The player chooses to play Checkers and another player is found.

Scenario:

1. The player clicks “Join Queue” and is matched up successfully with another player.
2. They are now at a screen where they can play Checkers and are notified that the game has started.
3. The players take turns and play the game until the game ends (win, lose, or draw).
4. After the game, they will be directed to the aftermath screen.
5. They can choose to return to the Checkers Home Menu or view the leaderboard.

Post Conditions: The game results are recorded and the aftermath screen is shown after the game is finished.

Exceptions:

- **E1:** The opponent disconnects, the system notifies the player, the game is ended.
- **E2:** The player’s connection is lost, the system stops the game.

Priority: High

When Available: After matchmaking.

Frequency of Use: Depends if the player can successfully match up with an opponent.

Channel to Actor: UI with real-time updates

Secondary actors: Opponent player

Channels to secondary actors: Network communication

Open Issues:

- Should the player be able to rematch with the opponent?

Use Case: No match found

Iteration: 1

Primary Actor: Player

Goal In Context: The player is notified that there is no match currently and they can retry or exit.

Preconditions: The player chooses to join a Checkers Game Queue.

Trigger: The player waits in the queue but the system could not find an opponent with in a certain time period.

Scenario:

1. The player chooses to join a Checkers Game Queue.
2. The matchmaking service searches for an opponent.
3. After a timeout period, the system still could not find an opponent.
4. The player is shown the “No Match Found” notification.
5. They have options to retry or exit back to the Checkers Home Menu.

Post Conditions: The player can try again or they are directed back to the Home Menu.

Exceptions:

- **E1:** There is an error while getting the player’s queue status, the system shows an error message and the player is redirected back to the Checkers Home Menu.

Priority: Medium

When Available: When there are low numbers of players.

Frequency of Use: Occasional, depends on the number of players.

Channel to Actor: UI notification

Secondary actors: None

Channels to secondary actors: N/A

Open Issues:

- What if there is no match for the player for a very long time and the player really wants to play?
- Should we include a function to have a game with AI?

Use Case: Checkers leaderboard

Iteration: 1

Primary Actor: Player

Goal In Context: The player would be able to see a screen with the leaderboard data for Checkers.

Preconditions:

- The player chooses Checkers from their profile Home Menu.
- The leaderboard functionality is available.

Trigger: The player is at the Checkers Home Menu and chooses “Leaderboard”.

Scenario:

1. The player chooses Checkers on their Profile Home Menu.
2. They want to see the leaderboard and select “Leaderboard”.
3. They are shown with the leaderboard for Checkers with the following data:
    - **Rank:** The player’s ranking based on win rate or score.
    - **Username:** The display name of the player.
    - **Total Wins:** The total number of games the player has won.
    - **Win Rate:** The percentage of games won out of total games played.
    - **Longest Win Streak:** The highest number of consecutive wins of the player.
    - **Games Played:** The total number of games the player has played.
    - **Last Played Date:** The date when the player last played.
    - **Moves to Win (Average):** The average number of moves the player used to win a game.

Post Conditions: The player has now viewed the leaderboard for Checkers with their stats and also other players’ stats, they can now go back to the Checkers Home Menu.

Exceptions:

- **E1:** The system fails to retrieve leaderboard data, the player is shown an error message.
- **E2:** The leaderboard functionality is not available, the player is informed and the have the choice to come back later or return to the Checkers Home Menu.

Priority: Medium

When Available: When the player is at the Checkers Home Menu.

Frequency of Use: Occasional, depends on if the player wants to check the leaderboard.

Channel to Actor: UI display of the leaderboard

Secondary actors: Leaderboard database/service

Channels to secondary actors: System-to-database communication

Open Issues:

- Should there be a friends system?
- Should the player be able to sort or filter the leaderboard?

Use Case: View Game Stats (Aftermath)

Primary Actor: Player

Goal in Context: Allow the player to view the stats of the game upon completion and compare results with the opponent before being sent back to the lobby/player profile

Preconditions: The player has completed a game of Connect 4, Checkers, or Tic Tac Toe

Trigger: The game ends, either through a win, loss, or draw

Scenario:

1. The game concludes, and the player is presented with a results screen
2. The results screen displays the outcome (win, lose, or draw) along with a comparison of their stats with the opponent's performance such as:
    - Number of moves played by each player
    - Total game time spent
    - Count of good moves and bad moves for each player
    - Highest streak of good or bad moves for each player
    - Updated ranks of both players
3. Before returning to the lobby/player profile, the player is shown a personal progress screen including:
    - Current rank
    - Progression to the next rank
    - Current win streak
    - Updated win/loss ratio
    - Any accolades earned in the session for example (Maximum Efficiency: finished the entire game without any bad moves, Fast Finisher: defeated opponent in less than one minute, etc)
4. The player is automatically redirected to the lobby/player profile to select the next match, or to view leaderboards, etc.

Post-Conditions: The player is returned to the lobby/player profile screen, and all game statistics and progress are accurately updated in the system.

Exceptions:

1. If the results screen fails to load, or the player disconnects prematurely, an error message is displayed, and the player is redirected back to the lobby/player profile.

Priority: High

When available: First Iteration

Frequency of Use: At all times, occurs at the end of every game session.

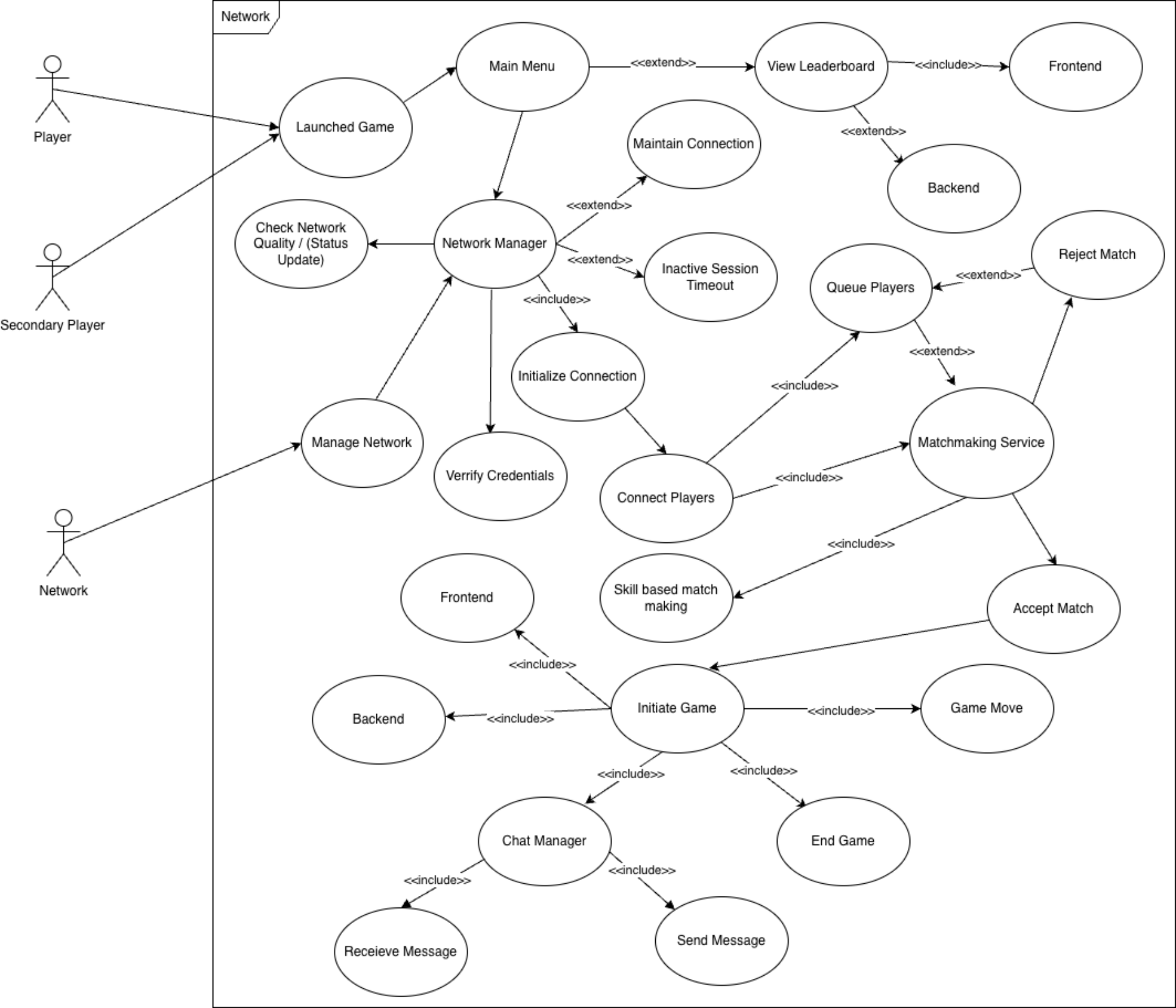
Channel to Actor: Player interacts with the game interface via a graphical user interface.

Secondary Actor: N/A

Channel to Secondary Actor: N/A

Open Issues:

1. Can this specific game's stats be accessed later in the player's profile for review after leaving the aftermath screen?
2. Can players manually choose to skip or delay the stats display and return directly to the lobby/player profile?


Network
Player
Secondary Player
Network
Launched Game
Main Menu
<<extend>>
View Leaderboard
<<include>>
Frontend
<<extend>>
Backend
Maintain Connection
<<extend>>
Check Network Quality / (Status Update)
Network Manager
<<extend>>
Inactive Session Timeout
<<include>>
Initialize Connection
Queue Players
<<extend>>
Reject Match
<<extend>>
<<include>>
Manage Network
Verrify Credentials
Connect Players
<<include>>
Matchmaking Service
<<include>>
Skill based match making
<<include>>
Accept Match
Frontend
<<include>>
Backend
<<include>>
Initiate Game
<<include>>
Game Move
<<include>>
<<include>>
Chat Manager
<<include>>
End Game
<<include>>
Receieve Message
<<include>>
Send Message

Use Case: Establish connection

Iteration: 1

Primary Actor: Player

Goal in Context: To establish a secure and stable connection with the network to access game services

Preconditions: Player has launched the game, Network Services are online and available

Trigger: Player initiates a connection attempt when starting the game

Scenario:

1. Player launches the game and reaches the main menu
2. Player selects an option that requires an online connection (all three of our games need it)
3. `NetworkManager` receives the player's request to connect
4. `NetworkManager` verifies player credentials and initiates a secure session (token? authentication method?)
5. `NetworkManager` checks the network quality and stable connection
6. Player is now connected to the network

Post Conditions: Player is successfully connected and ready to access online features (leaderboard, SBMM, chat)

Exceptions: Network is down and unavailable. Connection timeout: the network takes too long to respond.Poor connection quality.

Priority: High

When Available: 2nd iteration

Frequency of use: Every time a player initiates a connection or re-connects

Channel to Actor: GUI

Secondary Actor: Network (mostly `NetworkManager`)

Channels to Secondary Actor: Stub? (realistically an API call)

Open Issues: Say a player keeps having to reconnect, should there be a timer to stage intervals?

Use Case: Send Message

Iteration: 1

Actor: Player

Goal in Context: To allow a player to send a message to their opponent during a game session

Preconditions: Both players are connected to the same game session chat functionality is enabled for the game session

Trigger: Player initiates a message by typing into the in-game chat interface

Scenario:

1. Player opens the in-game chat interface
2. Player types a message in a text input box
3. Players send their message by clicking the "Send" button or pressing "Enter" on their keyboard.
4. `Network Manager` receives the message and forwards it to `ChatManager.`
5. `ChatManager` routes the message to the opponent
6. The message appears in the opponent's chat interface

Post Conditions: The message is received and displayed in the opponent's chat interface/window both players can see the updated chat log with the new message

Exceptions: A player disconnecting while in the process of sending their message. Network failure. Message blocking (inappropriate language).

Priority: High

When Available: 2nd iteration

Frequency of use: It varies as it depends on a player's communication preferences

Channel to Actor: In-game chat interface/window

Secondary Actor: Network - `NetworkManager` and `ChatManager`

Channels to Secondary Actor: N/A

Open Issues: Filtering chat which can be a long and heavy process

Use Case: Queue Entry

Iteration: 1

Actor: Player

Goal in Context: To allow a player to enter a matchmaking queue and wait for a compatible opponent to begin a game session

Preconditions: Both players are connected to the network. Matchmaking services are active and available

Trigger: Player selects an option to queue up matchmaking

Scenario:

1. Player navigates to the three different games they want to play in the GUI
2. Player chooses a game by selecting one
3. Player chooses to join matchmaking queue by clicking "find game" button in the GUI
4. `NetworkManager` sends a request to `MatchmakingService` to add the player to the queue
5. `MatchmakingService` places the player in a queue and begins searching for the right opponent based on SBMM criteria
6. Player receives a notification/status update to notify that they have found an opponent
7. The player is transitioned to the game session with their matched opponent

Post Conditions: Player is currently in an active match with their opponent or still remains in the queue if no match is available

Exceptions: Network failure. Timeout connection - if a player can't find a match within a certain amount of time. One of the players disconnects or leaves the matchmaking queue. Too many players in the matchmaking queue - a capacity limit

Priority: High

When Available: 2nd iteration

Frequency of use: Frequent as our games are all based on multiplayer

Channel to Actor: GUI interface

Secondary Actor: Network - `MatchmakingService`

Channels to Secondary Actor: Stub (API if we're being serious) calls and notifies from `NetworkManager` to `MatchmakingService`

Open Issues: We should define a time limit when waiting in queue before a timeout

Use Case: Status Update

Actor: Player

Goal in Context: To provide a player with updates regarding their current status within the game. For example, connection quality and matchmaking queue position

Preconditions: player is connected to the network and using the network features - SBMM, game chat, etc... `NetworkManager` or `MatchmakingService` is actively tracking player status

Trigger: A change in a player's status or an update from the network that needs to be communicated to the player

Scenario:

1. Player initiates an action that requires a status update - entering matchmaking queue or sending a message
2. `NetworkManager` detects a change in the payer's current activity - moving up a spot in matchmaking queue
3. `NetworkManager` sends a status update to the player's GUI/interface regarding their current activity - estimated wait time, current queue position, network status etc...
4. Player receives and views the update on their screen
5. Player may adjust their actions - exit queue if it's taking too long, latency issues / connection status

Post Conditions: The player is aware of their current status and makes a decision or not based on the updated information

Exceptions: Network failure. A player may disconnect before the status update is

received. Network latency might cause delay to status updates - potentially giving outdated or slow feedback.

Priority: Medium - it gives a nice experience for the players to know what their status is, but not as important as gameplay functionality

When Available: 3rd iteration

Frequency of use: It varies as it'll be based on network events or player actions

Channel to Actor: GUI - notification or a status window

Secondary Actor: Network - `NetworkManager` and `MatchmakingService`

Channels to Secondary Actor: Stub (API) calls for status changes

Open Issues: We have to determine which relevant statuses to update the player on the frequency and method of update delivery

Use Case: Game Move

Iteration: 1

Primary Actor: Player

Goal in Context: Enables the user to execute their move in the game and also maintains the proper game state synchronization between the two players.

Preconditions:

Player is matched with their suitable opponent.

The game starts and it is the player's turn to make their move.

The game state is properly synchronized and reflects the same at the other end on the player's side. Both the players are active and there is no disruption of connection between them.

Trigger: Player registers their move in the game and plays the turn.

Scenario:

1. The player has started the game
2. The game notifies the player it's their turn and is waiting for the player's move.
3. Player plays their and register's its move.

4. The system then checks if the move made was valid or not.
5. After the check is done, the move is then registered and made visible to both the players. The game state is updated and synchronized properly.

Postconditions:

The game state is updated and maintained

After the move:

If the move made by the player results in a win/loss/draw, the game is over and the state is updated accordingly.

Otherwise the turn then shifts to the opponent.

Exceptions:

Invalid/Not accepted move is played.

Delay in response thereby affecting the game state and synchronization.

Either of the players gets disconnected while playing their move.

Priority: High

When Available: 3rd Iteration

Frequency of Use: Very frequent (as this will allow the users to play eventually)

Channel to actor: Player plays the game with just simple mouse clicks/keys as per the game functionality.

Use Case: End game

Iteration: 1

Primary Actor: Network

Goal in Context: Terminates a game session, whenever a winning move is played thereby resulting the game to end and then notifies both the players about the results.

Preconditions:

Both the players are in an active game session.

Either of the players have played a winning move resulting in game termination.

Either of the players have played a move that resulted in a draw, the game also terminates in that case and notifies the player.

Either of the players have forfeited. A forfeit will terminate the game and the player who forfeited will automatically lose.

Trigger: The system detects that either of the players has played a winning/losing/drawing move.

Scenario:

1. The system receives the last move from the player.
2. The move is then verified and checked.
3. The game state is updated and synchronized for both the players.
4. The system then does all the calculations and detects that it is a winning move played by either of the players.
5. The system then displays the game results to both the players and indicates that the game is over. The system then updates the leaderboard as per the game results and player's stats.

Postconditions:

The game session is terminated properly.

Player rankings are updated accordingly in the leaderboard after every game played.

Exceptions:

Connection disruption may occur due to which even after playing a winning move, the system couldn't detect that and the game doesn't end.

The move played that wasn't checked properly might also lead to end the game abruptly.

The game stats for the players weren't updated properly.

Priority: High

When Available: 3rd Iteration

Frequency of Use: Very frequent (after every game session)

Channel to actor: The system coordinates the game ending process automatically after it detects the move played.

Use Case: View Leaderboard

Iteration: 1

Primary Actor: Player

Goal in Context: Allows players to check the leaderboard in order to view their current standings and rankings for other players.

Preconditions:

- The player is logged in and has played a sufficient number of games in order to be visible on the leaderboard.

- Leaderboard is available to retrieve the rankings for all players as per their gaming history and stats.

Trigger: Whenever the player requests to view the leaderboard.

Scenario:

1. The player has finished their game session.
2. The player then requests to view the leaderboard.
3. The request is received by the system and thereby it fetches all the current top rankings from the database.
4. The GUI interface receives the requested data and displays it as a leaderboard.
5. The players therefore can then view their current standings and compare it with their opponents too.

Postconditions:

- The player is able to successfully retrieve the leaderboard data and view their rankings. The rankings calculated are correct and that it was updated as per the latest game played by the player.

Exceptions:

- There was a disruption in connection which caused miscalculation in rankings thereby the ranking displayed was incorrect.
- The leaderboard rankings were not updated after each game played.

Priority: High

When Available: 3rd iteration

Frequency of Use: Quite a few sometimes.(Depends on the player)

Channel to actor: Player interacts with the GUI just normally using the mouse in order to navigate and view the leaderboard.

Use Case: Initialize Connection

Iteration: 1

Primary Actor: Network

Goal in Context: Establish the connection between the player and the server, which allows for access to matchmaking and chat services

Preconditions: The application has been launched and the network manager is ready to accept connections

Trigger: The player starts the connection by logging in to the application

Scenario:

1. Player requests a connection to the network by trying to log in
2. NetworkManager verifies the players log in information
3. If successful NetworkManager creates a connection for the player
4. Player is notified that the connection was successful, and now has access to online services

Post Conditions:

- There is a stable connection between the network and the player

Exceptions:

- UserID or password is incorrect

- NetworkManager fails to establish a connection

Priority: Essential

When Available: 1st Iteration

Frequency of Use: Very high

Channel to Actor: User interface, network connection

Use Case: Maintain Connection

Iteration: 1

Primary Actor: Network

Goal in Context: Ensure the connection between the player and the server remains active and stable for access to online services

Preconditions:

- The player has initially connected to the network successfully
- NetworkManager is actively monitoring the connection

Trigger: The player is actively using the online services within the application

Scenario:

1. Player successfully initiates a connection
2. NetworkManager periodically checks the status of the connection
3. If the connection is stable, NetworkManager continues communication between the player and the network
4. If the connection is unstable NetworkManager attempts to reestablish a stable connection
5. Player will receive a notification if their connection experiences any issues

Post Conditions:

- The connection remains active and stable

Exceptions:

- Connection is completely lost
- Reconnection fails after several attempts to reconnect

Priority: Essential

When Available: 1st iteration

Frequency of Use: Very high

Channel to Actor: User interface, network connection, connection notifications

Use Case: Manage Network

Iteration: 1

Primary Actor: Network

Goal in Context: To Ensure that all connections are being managed. Which includes initialization, maintenance, and termination of player connections to maintain a stable connection to the network

Preconditions:

- The servers are active, and NetworkManager is functioning
- Online services (Matchmaking, chat, game sessions) are enabled, and accessible to players

Trigger: The player initiates any kind of network activity

Scenario:

1. The NetworkManager is active and listening for network requests from players
2. A request is received and NetworkManager checks for network availability
3. NetworkManager fulfills the players request
4. NetworkManager continues to monitor for requests and routing to different services

Post Conditions:

- All active connections are monitored, maintained, and closed as needed
- Network resources are efficiently allocated and released
- Players experience minimal interruptions in their connection to network services

Exceptions:

- NetworkManager fails to establish a connection insufficient resources
- NetworkManager encounters unexpected latency during an active session
- Disconnection occurs due to a network error

Priority: Essential

When Available: 1st iteration

Frequency of Use: Very high

Channel to Actor: Network connection, server logs, graphical interface

Use Case: Session Timeout/Inactive

Iteration: 2

Primary Actor: Network

Goal in Context: End inactive sessions to free up network resources

Preconditions:

- The player is connected to the network and their session has been idle for 10 minutes

- NetworkManager is functioning

Trigger: The current players session has been idle (no activity) for 10 minutes

Scenario:

1. A players session has been idle for 10 minutes
2. An inactivity warning is displayed to the player
3. If the player remains inactive NetworkManager terminates their session
4. A notification that their session has been terminated is displayed to the player

Post Conditions:

- Player status is updated to offline

Exceptions:

- Player resumes activity during the warning period, cancelling the timeout.

- NetworkManager fails to terminate the session due to a network error.

Priority: Medium

When Available: 2nd iteration

Frequency of Use: Low to medium

Channel to actor: Graphical interface and server logs

Use Case: Accept Match

Iteration: 1

Primary Actor: Player

Goal in Context: Allows players to accept the match found by the matchmaking service based on rankings of the players and start their game.

Preconditions:

- Player has started the game and is already in the queue for matchmaking.
- The matchmaking service finds a suitable opponent for the player.
- No disruption of connection between the player and the game.

Trigger: Player is notified about the suitable match found and is then selects the option to Accept the match.

Scenario:

1. The player is on the queue for matchmaking.
2. The matchmaking service finds a suitable opponent for the player.
3. The system notifies the player about the match found and displays the ranking and stats for the opponent.
4. The system then prompts the player if the player wants to accept the match or not.
5. If both the players accept their respective matches then, a game session is created and both the players are transitioned to the game screen.

Post Conditions:

- Both the players start their game session.

Exceptions:

- The matchmaking times out before the player accepts it.

- There's a network connection disruption and the players get disconnected.

- The opponent rejects the match.

Priority: High as this will actually start the game.

When Available: 3rd iteration

Frequency of Use: Very frequent depending on the number of players in queue.

Channel to actor: The player interacts by just clicking on the accept button with the mouse.

Use Case: Reject Match

Iteration: 1

Primary Actor: Player

Goal in Context: Allows players to reject the match found by the matchmaking service based on rankings of the players and start their game.

Preconditions:

- Player has started the game and is already in the queue for matchmaking.

- The matchmaking service finds a suitable opponent for the player.

- No disruption of connection between the player and the game.

Trigger: Player is notified about the suitable match found and then selects the option to reject the match.

Scenario:

1. The player is on the queue for matchmaking.
2. The matchmaking service finds a suitable opponent for the player.
3. The system notifies the player about the match found and displays the ranking and stats for the opponent.
4. The system then prompts the player if the player wants to accept the match or not.

5. If both the players reject their respective matches then, no game session is created and both the players return back to the matchmaking queue.

Post Conditions:

- No game session is created.

- Both the players are notified about the same and they both return back to the matchmaking queue.

Exceptions:

- There’s a network connection disruption and the players get disconnected.

- The system fails to detect match rejection by the player and starts the game instead.

Priority: High

When Available: 3rd iteration

Frequency of Use: Very frequent depending on the number of players in queue.

Channel to actor: The player interacts by just clicking on the reject button with the mouse.